DA2009 - Group Project - IRCSL Ethical Web Scraping

DA 2009 - Data Collection Methods II
Group 3

Name	Student ID	Phase
Chandrasekaran Sivaneswaran	25ada147	Phase 3 - Brokers and Loss Adjusters
Zainul Abdeen Athif	25ada023	Phase 4 - Document Engine
V. K. Rajapakse	25ada139	Phase 1 - Static Pages
P. P. M. Subasinsani	25ada029	Phase 2 - Insurance Companies

Phase 1 - Ethical Setup, Site Mapping and Static Pages

Done by: V. K. Rajapakse (25ada139)
Website: https://ircsl.gov.lk (Insurance Regulatory Commission of Sri Lanka)

In this phase we set up the ethical rules that the whole group follows (robots.txt, User-Agent and delays), map the website using its sitemap, and scrape two static pages: **About Us** and **Contact Us** with Requests, BeautifulSoup and pandas.

1. Setup

We use `requests` to download pages, `BeautifulSoup` to read the HTML and `pandas` to store the results. `urllib.robotparser` reads robots.txt files.

Our scraping rules, used by the whole group:

- a **User-Agent** that says who we are, instead of the default `python-requests` one
- a **3-second delay** after every request so we do not overload the server
- a **timeout** of 20 seconds so the program does not wait forever

In [1]:
```
import os
import time
import urllib.robotparser

import requests
import pandas as pd
from bs4 import BeautifulSoup

USER_AGENT = "DA2009-StudentProject/1.0 (BSc Applied Data Analytics coursework; academic use only)"
HEADERS = {"User-Agent": USER_AGENT}
DELAY_SECONDS = 3
TIMEOUT = 20

BASE_URL = "https://ircsl.gov.lk"
ROBOTS_URL = BASE_URL + "/robots.txt"
ABOUT_US_URL = BASE_URL + "/about-us/"
CONTACT_US_URL = BASE_URL + "/contact-us/"

os.makedirs("data/raw", exist_ok=True)
os.makedirs("data/processed", exist_ok=True)
```

2. Reading robots.txt

robots.txt tells crawlers which parts of a website they can and cannot visit. We print it, load it into `RobotFileParser`, and use `can_fetch()` to check our URLs. The site

only blocks `/wp-admin/` (the WordPress admin area), so the public pages we need are allowed.

In [2]:
```
robots_text = requests.get(ROBOTS_URL, headers=HEADERS, timeout=TIMEOUT).text
print(robots_text)

rp = urllib.robotparser.RobotFileParser()
rp.set_url(ROBOTS_URL)
rp.read()

crawl_delay = rp.crawl_delay(USER_AGENT)
print("Crawl-delay given by the website:", crawl_delay, "| we use", DELAY_SECONDS, "seconds")
time.sleep(DELAY_SECONDS)

test_urls = [ABOUT_US_URL, CONTACT_US_URL, BASE_URL + "/wp-admin/"]
pd.DataFrame({"url": test_urls, "allowed": [rp.can_fetch(USER_AGENT, u) for u in test_urls]})
```

```
User-agent: *
Disallow: /wp-admin/
Allow: /wp-admin/admin-ajax.php

Sitemap: https://ircsl.gov.lk/wp-sitemap.xml

Crawl-delay given by the website: None | we use 3 seconds
```

Out[2]:

	url	allowed
0	https://ircsl.gov.lk/about-us/	True
1	https://ircsl.gov.lk/contact-us/	True
2	https://ircsl.gov.lk/wp-admin/	False

3. A function to fetch pages politely

All pages are downloaded with the same function, `polite_get()`. It:

1. checks robots.txt before the request
2. sends the request using a **Session**, which keeps the connection open and sends our User-Agent every time
3. checks the **HTTP status code**: 200 means OK, 404 means not found, 429 means too many requests, and 5xx means a server error
4. waits 3 seconds after every request

If the page downloads correctly, we save a copy of the HTML in `data/raw` so we do not need to download it again.

In [3]:
```
session = requests.Session()
session.headers.update(HEADERS)


def polite_get(url):
    if not rp.can_fetch(USER_AGENT, url):
        print("Not allowed by robots.txt:", url)
        return None

    try:
        response = session.get(url, timeout=TIMEOUT)
    except requests.exceptions.RequestException as error:
        print("Request failed:", url, error)
        return None

    time.sleep(DELAY_SECONDS)
    status = response.status_code

    if status == 200:
        print("200 OK:", url)
        file_name = url.rstrip("/").split("/")[-1]
        if "." not in file_name:
            file_name = file_name + ".html"
        with open("data/raw/" + file_name, "w", encoding="utf-8") as f:
            f.write(response.text)
```

```
        return response.text
    elif status == 404:
        print("404 Page not found:", url)
    elif status == 429:
        print("429 Too many requests, waiting longer:", url)
        time.sleep(DELAY_SECONDS * 5)
    elif status >= 500:
        print(status, "Server error:", url)
    else:
        print(status, "Unexpected status:", url)
    return None
```

4. Mapping the website with the sitemap

The last line of robots.txt gives the sitemap, `wp-sitemap.xml`. A sitemap is an XML file that lists the pages of a website, so we can find pages without crawling every link. The main sitemap is a list of smaller sitemaps; we only open the one for English pages. Every URL is inside a `<loc>` tag, and because the file is XML we use the `"xml"` parser.

The rest of the group used this site map to find their pages. We also found that there are **no separate Board of Directors or Management Team pages**; those people are on the About Us page.

In [4]:
```
sitemap_url = rp.site_maps()[0]
index_soup = BeautifulSoup(polite_get(sitemap_url), "xml")
print("Smaller sitemaps listed in", sitemap_url, ":", len(index_soup.find_all("loc")))

pages_soup = BeautifulSoup(polite_get(BASE_URL + "/wp-sitemap-posts-page-1.xml"), "xml")
page_rows = []
for loc in pages_soup.find_all("loc"):
    url = loc.get_text(strip=True)
    path = url.replace(BASE_URL, "")
    section = path.strip("/").split("/")[0] or "(home)"
    page_rows.append({"page_url": url, "path": path, "section": section})

site_pages = pd.DataFrame(page_rows)
print("English pages in the sitemap:", len(site_pages))

for target in ["about-us", "board-of-directors", "management-team", "contact-us"]:
    print(target, "found:", site_pages["path"].str.contains(target).any())

site_pages.head(10)
```

```
200 OK: https://ircsl.gov.lk/wp-sitemap.xml
Smaller sitemaps listed in https://ircsl.gov.lk/wp-sitemap.xml : 27
200 OK: https://ircsl.gov.lk/wp-sitemap-posts-page-1.xml
English pages in the sitemap: 90
about-us found: True
board-of-directors found: False
management-team found: False
contact-us found: True
```

Out[4]:

	page_url	path	section
0	https://ircsl.gov.lk/insurance-legislation/reg...	/insurance-legislation/regulation-of-insurance...	insurance-legislation
1	https://ircsl.gov.lk/insurance-sector/industry...	/insurance-sector/industry-performance/	insurance-sector
2	https://ircsl.gov.lk/instructions-and-guidelin...	/instructions-and-guidelines/licensing-guide/	instructions-and-guidelines
3	https://ircsl.gov.lk/publications/press-releases/	/publications/press-releases/	publications
4	https://ircsl.gov.lk/insurance-sector/industry...	/insurance-sector/industry-performance/previou...	insurance-sector
5	https://ircsl.gov.lk/instructions-and-guidelin...	/instructions-and-guidelines/licensing-guide/i...	instructions-and-guidelines
6	https://ircsl.gov.lk/quick-links/regulatory-re...	/quick-links/regulatory-reporting/compliance-w...	quick-links
7	https://ircsl.gov.lk/quick-links/regulatory-re...	/quick-links/regulatory-reporting/compliance-w...	quick-links
8	https://ircsl.gov.lk/quick-links/regulatory-re...	/quick-links/regulatory-reporting/compliance-w...	quick-links
9	https://ircsl.gov.lk/quick-links/regulatory-re...	/quick-links/regulatory-reporting/compliance-w...	quick-links

5. About Us page (BeautifulSoup)

Using Inspect in the browser, we saw that each person is shown as a card, not as a table row:

```
<div class="tc_team-member ...">
    <div class="tc_overlay">
        <h3>Dr. W. A. R. De Mel</h3>
        <h4 class="tc_job_role">Chairman</h4>
    </div>
</div>
```

The cards do not say which group the person belongs to. The groups are only separated by headings between the cards (`[8 cards] -> Directors -> [7 cards] -> Assistant Directors -> [12 cards]`).

So `select("h3, div.tc_team-member")` gives the headings and cards **in page order**, and we remember the last heading we passed. `clean_text()` removes extra spaces and `to_null()` turns the "-" used for vacant posts into `None` .

In [5]:

```
about_soup = BeautifulSoup(polite_get(ABOUT_US_URL), "html.parser")
elements = about_soup.find("div", class_="pill-content").select("h3, div.tc_team-member")


def clean_text(text):
    text = str(text).replace("\xa0", " ")
    return " ".join(text.split())


def to_null(text):
    text = clean_text(text)
    return None if text in ["", "-", "–"] else text


scraped_at = time.strftime("%Y-%m-%d %H:%M:%S")
current_group = "Commission Member / Senior Management"
team_rows = []

for el in elements:
    if el.name == "h3" and clean_text(el.get_text()) in ["Directors", "Assistant Directors"]:
        current_group = clean_text(el.get_text())[:-1]     # "Directors" -> "Director"
        continue
    if el.name == "div":
        name_tag = el.select_one(".tc_overlay h3")
        title_tag = el.select_one(".tc_overlay h4.tc_job_role")
        team_rows.append({
            "name": to_null(name_tag.get_text()) if name_tag else None,
            "title": to_null(title_tag.get_text()) if title_tag else None,
            "group": current_group,
            "source_url": ABOUT_US_URL,
            "scraped_at": scraped_at,
        })

about_df = pd.DataFrame(team_rows)
print("People on the About Us page:", len(about_df))
print(about_df["group"].value_counts().to_dict())
about_df.head(10)
```

```
200 OK: https://ircsl.gov.lk/about-us/
People on the About Us page: 27
{'Assistant Director': 12, 'Commission Member / Senior Management': 8, 'Director': 7}
```

Out[5]:

	name	title	group	source_url	scraped_at
0	Dr. W. A. R. De Mel	Chairman	Commission Member / Senior Management	https://ircsl.gov.lk/about-us/	2026-10-09 11:58:37
1	Mr. A.N. Hapugala	Ex-officio Member / Deputy Secretary to the Tr...	Commission Member / Senior Management	https://ircsl.gov.lk/about-us/	2026-10-09 11:58:37
2	Dr. Chandranath Amarasekara	Ex-officio Member / Deputy Governor, Central B...	Commission Member / Senior Management	https://ircsl.gov.lk/about-us/	2026-10-09 11:58:37
3	Mrs. K.A.P. De Silva	Ex-Officio Member (Registrar General of Compan...	Commission Member / Senior Management	https://ircsl.gov.lk/about-us/	2026-10-09 11:58:37
4	Mr. R.C. Kotte Hewa	Member	Commission Member / Senior Management	https://ircsl.gov.lk/about-us/	2026-10-09 11:58:37
5	Mr. G.A.S. Jayaratna	Member	Commission Member / Senior Management	https://ircsl.gov.lk/about-us/	2026-10-09 11:58:37
6	Mr. S. Waidyarathne	Member	Commission Member / Senior Management	https://ircsl.gov.lk/about-us/	2026-10-09 11:58:37
7	Mrs. Damayanthi Fernando	Director General	Commission Member / Senior Management	https://ircsl.gov.lk/about-us/	2026-10-09 11:58:37
8	Mr. Sathiesh Kumar	Director (Finance)	Director	https://ircsl.gov.lk/about-us/	2026-10-09 11:58:37
9	Mrs. Prabhashini Samarakoon	Director (Investigation)	Director	https://ircsl.gov.lk/about-us/	2026-10-09 11:58:37

6. Contact Us page (pd.read_html)

The Contact Us page has a normal HTML table (Name, Position, Email, Phone). For a simple table like this, `pd.read_html()` reads it in one line, so we did not need to loop over the cells with BeautifulSoup. We read the copy of the page saved by `polite_get()`.

In [6]:

```
polite_get(CONTACT_US_URL)
contact_df = pd.read_html("data/raw/contact-us.html")[0]

contact_df.columns = [clean_text(c).lower() for c in contact_df.columns]
for column in contact_df.columns:
    contact_df[column] = contact_df[column].map(to_null)
contact_df["source_url"] = CONTACT_US_URL
contact_df["scraped_at"] = scraped_at

print("Rows from Contact Us:", len(contact_df))
contact_df.head(10)
```

```
200 OK: https://ircsl.gov.lk/contact-us/
Rows from Contact Us: 20
```

Out[6]:

	name	position	email	phone	source_url	scraped_at
0	Insurance Regulatory Commission of Sri Lanka	General Contact	info@ircsl.gov.lk	+94-11-2396184 – 9	https://ircsl.gov.lk/contact-us/	2026-10-09 11:58:37
1	Dr. W.A.R. De Mel	Chairman	chairman@ircsl.gov.lk	+94-11-2396180	https://ircsl.gov.lk/contact-us/	2026-10-09 11:58:37
2	Mrs. Damayanthi Fernando	Director General	dg@ircsl.gov.lk	+94-11-2396181	https://ircsl.gov.lk/contact-us/	2026-10-09 11:58:37
3	Mr. Sathiesh Kumar	Director – Finance	df@ircsl.gov.lk	+94-11-2396193	https://ircsl.gov.lk/contact-us/	2026-10-09 11:58:37
4	Mrs. Prabhashini Samarakoon	Director – Investigation	di@ircsl.gov.lk	+94-11-2396194	https://ircsl.gov.lk/contact-us/	2026-10-09 11:58:37
5	Mrs. Deepika Navarathne	Director – Market Development and External Rel...	dmder@ircsl.gov.lk	+94-11-2396184-9 (Ext. 138)	https://ircsl.gov.lk/contact-us/	2026-10-09 11:58:37
6	Mr. G Rajan Nirubasingham	Director – Legal and Enforcement	dl@ircsl.gov.lk	+94-11-2396182	https://ircsl.gov.lk/contact-us/	2026-10-09 11:58:37
7	Mrs. Chamari Heenatigala	Director Market Intermediary Licensing & Super...	dmi@ircsl.gov.lk	+94-11-2396184-9	https://ircsl.gov.lk/contact-us/	2026-10-09 11:58:37
8	Mrs. Lasanthi Thotahewa	Director – Administration	da@ircsl.gov.lk	+94-11-2396184-9	https://ircsl.gov.lk/contact-us/	2026-10-09 11:58:37
9	Mrs. Kalani Hettiarachchi	Director – Supervision	ds@ircsl.gov.lk	+94-11-2396183	https://ircsl.gov.lk/contact-us/	2026-10-09 11:58:37

7. Saving the results

In [7]:
```
site_pages.to_csv("data/processed/site_map_pages.csv", index=False)
about_df.to_csv("data/processed/about_us_team.csv", index=False)
about_df.to_json("data/processed/about_us_team.json", orient="records", indent=2)
contact_df.to_csv("data/processed/contact_us_directory.csv", index=False)

print("Pages in site map:", len(site_pages))
print("About Us people:", len(about_df))
print("Contact Us rows:", len(contact_df))
```

```
Pages in site map: 90
About Us people: 27
Contact Us rows: 20
```

8. Ethical considerations

- We read robots.txt with code and checked every URL before requesting it.
- We waited 3 seconds after every request, and our User-Agent says that this is a student project.
- We only made 5 requests: robots.txt, two sitemaps, About Us and Contact Us. We saved the HTML so we did not need to download the pages again.
- We only used public pages and did not try to access the admin area.
- The names, emails and phone numbers are official contact details published by IRCSL. We only use them for this assignment.
- Every row keeps the page it came from (`source_url`) and when it was scraped (`scraped_at`).